Real-Time PPE Detection

A Computer Vision System for Construction Site Safety

Mini Project - Real-Time Computer Vision System

Submitted by

Name	Roll No.
Shantanu Vaidya	B2-20
Vineet Shah	B2-28
Mohammad Ejaz Mehendi	B2-25

Under the guidance of

Sweta Jain

Abstract. This project builds a real-time system that detects personal protective equipment (PPE) on construction workers, and warns when a hardhat, vest or mask is missing. It works on images, video files and live camera streams. We used the public Construction Site Safety dataset (2,801 images, 10 classes), checked and cleaned its labels, and trained a YOLOv8n object detector. The model reached a precision of 0.786, a recall of 0.530 and an mAP@0.5 of 0.600 at about 25 frames per second on a laptop CPU. We compared it with four state-of-the-art YOLO models and with published results on the same dataset. The system is served through a Streamlit web app that runs on laptops and mobile phones.

1. Introduction

Construction is one of the most dangerous industries. Hardhats, safety vests and masks protect workers from head injuries, being struck by vehicles and dust. Today supervisors check this by walking around the site, which is slow and cannot be done all the time. A camera with a computer vision model can check every frame automatically. YOLO detectors find and classify many objects in one pass, so they are fast enough for live video.

Problem statement. Develop a system that detects workers, their PPE and missing PPE in real time from images, video files and live camera streams, shows a warning when PPE is missing, and measures its accuracy with precision, recall and mAP.

Objectives.

1. Clean and check a public PPE dataset and prepare the training, validation and test splits.
2. Train a YOLOv8n detector and evaluate it with precision, recall, mAP and speed (FPS).
3. Compare it with state-of-the-art YOLO models and with published results.
4. Build a web interface that works on computers and mobile phones.

2. Literature Review

We reviewed six recent papers (2024): four on new real-time object detectors and two on PPE detection at construction sites.

2.1 Real-time object detectors

YOLOv9 [5] adds Programmable Gradient Information (PGI) and a light backbone called GELAN, which use parameters more efficiently. **RT-DETR** [3] is a transformer detector that does not need non-maximum suppression (NMS); it reaches 53.1% AP on COCO at 108 FPS on a T4 GPU. **YOLOv10** [4] also removes NMS and trims extra layers; its small model is about 1.8x faster than RT-DETR-R18 at similar accuracy. **YOLO11** [6] adds C3k2 blocks, SPPF pooling and a C2PSA attention block to get better accuracy with fewer parameters.

2.2 PPE detection on construction sites

Ciftci, Turkdamar and Ozturk [1] used the same Construction Site Safety dataset as this project. Training for 20 epochs at 640 px, YOLOv5n reached 0.66 mAP@0.5 and YOLOv8n reached 0.727. **Feng, Miao and Zheng** [2] trained YOLOv5 and YOLOv8 models on a much larger dataset of 103,500 images; YOLOv8s was best with 84.0% mAP@0.5, 85.0% precision and 60.5% recall.

Table 1. Comparison of reviewed papers.

Paper	Year	Task / data	Models	Key result	Relevance
Wang, Yeh, Liao [5]	2024	General, COCO	YOLOv9, GELAN	Better parameter use	Efficient YOLO design
Zhao et al. [3]	2024	General, COCO	RT-DETR R50/R101	53.1% AP at 108 FPS (T4)	Transformer alternative
Wang et al. [4]	2024	General, COCO	YOLOv10 N-X	1.8x faster than RT-DETR-R18	NMS-free, lower latency
Khanam, Hussain [6]	2024	Architecture review	YOLO11 N-X	C3k2, SPPF, C2PSA	Newest YOLO version
Ciftci et al. [1]	2024	PPE, same dataset	YOLOv5n, YOLOv8n	YOLOv8n 0.727 mAP@0.5	Direct comparison
Feng et al. [2]	2024	PPE, 103,500 images	YOLOv5n/s, v8n/s	YOLOv8s 84.0% mAP@0.5	Same model families

Observation. In both PPE studies recall is lower than precision (for example 60.5% vs 85.0%), which means models miss more objects than they wrongly detect. For safety, a missed violation is the bigger risk, so this project reports recall alongside mAP.

3. Methodology

The project follows six steps (Figure 1). Each step is a separate Python script.

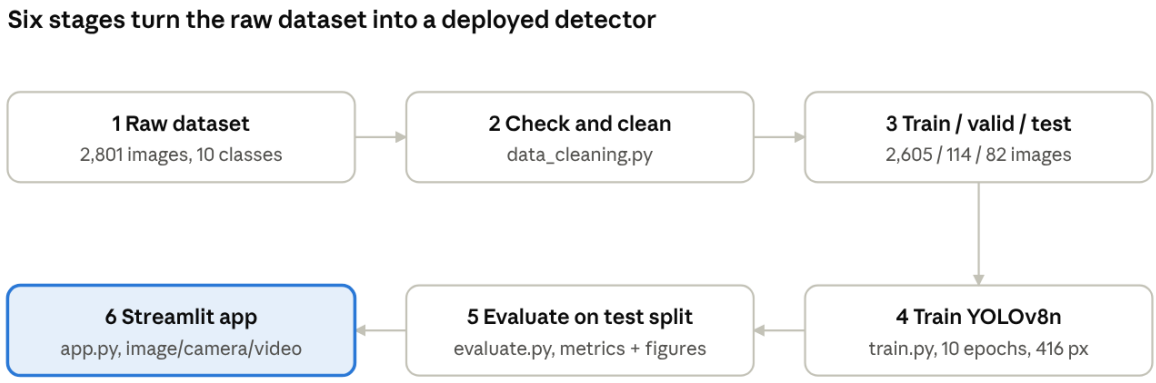


Figure 1. Project pipeline from raw dataset to web app.

3.1 Dataset

The Construction Site Safety dataset [8] (Roboflow, YOLO format) has 2,801 images with 10 classes: Hardhat, Mask, NO-Hardhat, NO-Mask, NO-Safety Vest, Person, Safety Cone, Safety Vest, machinery and vehicle. The NO-* classes mean an item is missing and trigger the warning in the app. Each label line holds a class number and the box centre, width and height, scaled to 0-1.

3.2 Model

We used **YOLOv8n** (nano), the smallest and fastest YOLOv8 model (3.2 M parameters [7]). It looks at the whole image once and predicts boxes and classes together, which makes it fast enough for live video on a laptop. We started from weights pre-trained on the COCO dataset and fine-tuned them on the PPE images (transfer learning).

3.3 Training setup

Table 2. Training settings.

Setting	Value
Model and initial weights	YOLOv8n, COCO pre-trained
Epochs	10
Image size / batch size	416 x 416 px / 16
Optimiser	Ultralytics default (auto)
Augmentation	Mosaic, horizontal flip, colour changes, scaling (Ultralytics defaults)
Hardware	Laptop CPU, AMD Ryzen 7 7840HS (no GPU)
Training time	0.77 hours

3.4 Evaluation metrics

A detection is counted as correct when its box overlaps the true box by at least 50% (IoU >= 0.5). **Precision** = TP/(TP+FP) tells how many detections were correct. **Recall** = TP/(TP+FN) tells how many real objects were found. **mAP@0.5** is the average precision over all 10 classes at IoU 0.5, and **mAP@0.5:0.95** averages it over stricter overlap levels. **FPS** (frames per second) measures speed.

4. Data Cleaning and Preprocessing

4.1 Data cleaning

The script data_cleaning.py checks every image and label file. The results are in Table 3.

Table 3. Data quality checks.

Check	Result
Corrupt or unreadable images	0 of 2,801
Images without labels / labels without images	0
Invalid class numbers or box coordinates	0
Duplicate boxes in one file	2 (removed)
Images with no objects	24 (kept as background examples)

4.2 Data splitting

We used the dataset's train, validation and test split. The model learns from the training set, the validation set is used to pick the best epoch, and the test set is used only for the final results.

Table 4. Dataset split.

Split	Images	Share	Boxes
Train	2,605	93%	36,893
Validation	114	4%	697
Test	82	3%	760

4.3 Class balance and preprocessing

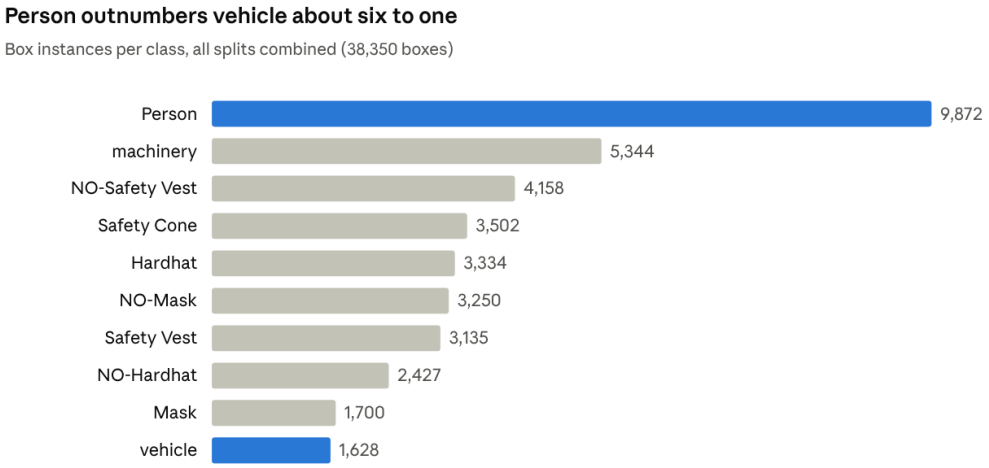


Figure 2. Number of boxes per class; Person outnumbers vehicle about six to one.

The classes are not balanced (Figure 2): Person has 9,872 boxes but vehicle only 1,628, so rare classes are expected to be harder to detect. Images are 640 x 640 px. During training, YOLO scales pixel values to 0-1 and applies augmentation: mosaic (four images combined), flips, colour changes and scaling.



Figure 3. Sample images with their labelled boxes (red = missing PPE class).

5. Results and Discussion

5.1 Results

On the 82 test images, YOLOv8n achieved the results in Table 5, at about 25 FPS on the laptop CPU, which is fast enough for live video without a GPU.

Table 5. YOLOv8n results on the test set.

Precision	Recall	mAP@0.5	mAP@0.5:0.95	Speed (CPU)
0.786	0.530	0.600	0.299	~25 FPS

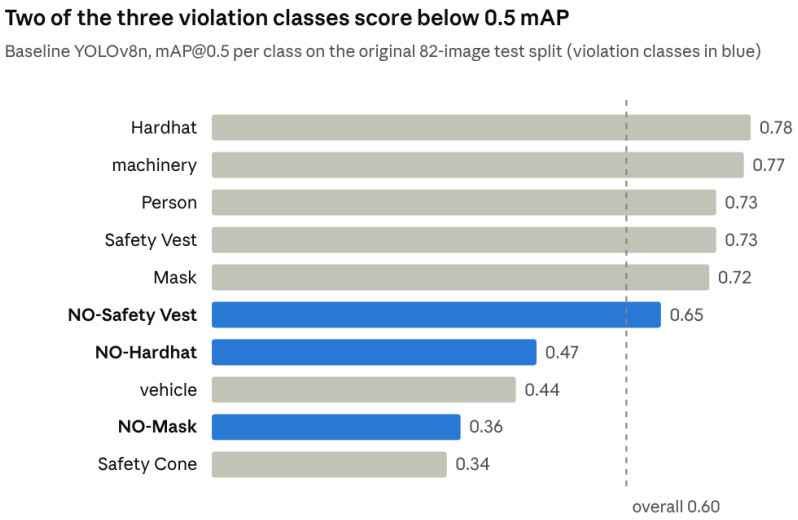


Figure 4. mAP@0.5 per class; two of the three missing-PPE classes score below 0.5.

The model detects Hardhat (0.78), machinery (0.77), Person and Safety Vest (0.73) well. The weakest classes are NO-Mask (0.36), NO-Hardhat (0.47), Safety Cone (0.34) and vehicle (0.44). A missing item is harder to see than a present one, and cones and vehicles are often small or rare. Precision (0.786) is higher than recall (0.530): the model is usually right when it detects something, but it misses some objects. Lowering the confidence threshold in the app finds more objects at the cost of some false detections.

5.2 Limitations

- **Look-alikes:** an ordinary cap is sometimes detected as a Hardhat, because the dataset has few people wearing normal caps.
- **Small objects:** cones, vehicles and far-away workers are often missed.
- **Short training:** only 10 epochs on a CPU; more epochs on a GPU should improve accuracy.
- **Scope:** the system checks that PPE is present, not that it is worn correctly.

6. State-of-the-art Models Comparison

We compare our model in two ways: with four state-of-the-art YOLO models on the standard COCO benchmark (Table 6), and with published PPE detection results (Table 7).

Table 6. State-of-the-art YOLO models on the COCO benchmark (640 px, from the Ultralytics documentation [7]).

Model	Released	Key design	Params (M)	FLOPs (B)	COCO mAP @0.5:0.95
YOLOv5s (yolov5su)	2020 (u head 2023)	CSPDarknet backbone, FPN+PAN neck	9.1	24.0	43.0
YOLOv8n (used here)	Jan 2023	Anchor-free head, C2f blocks	3.2	8.7	37.3
YOLOv8s	Jan 2023	YOLOv8n design, wider and deeper	11.2	28.6	44.9
YOLO11n	Sep 2024	C3k2 blocks, SPPF, C2PSA attention	2.6	6.5	39.5

Larger models (YOLOv8s, YOLOv5s) are more accurate but need three to four times more computation. YOLO11n is both smaller and more accurate than YOLOv8n, so it is a good next model to try. We chose YOLOv8n because it is fast and widely supported.

Table 7. PPE detection results compared with published work.

Study	Model	Dataset	Training	Precision	Recall	mAP@0.5
Ciftci et al. [1]	YOLOv5n	Construction Site Safety (same)	20 epochs, 640 px, GPU	0.759	0.592	0.660
Ciftci et al. [1]	YOLOv8n	Construction Site Safety (same)	20 epochs, 640 px, GPU	0.871	0.654	0.727
Feng et al. [2]	YOLOv8s	Roboflow, 103,500 images, 9 classes	200 iterations, GPU	0.850	0.605	0.840
This work	YOLOv8n	Construction Site Safety (same)	10 epochs, 416 px, CPU	0.786	0.530	0.600

On the same dataset, our YOLOv8n reaches 0.600 mAP@0.5, compared with 0.727 for YOLOv8n in [1]. The difference comes from training: we used half the epochs (10 vs 20), smaller images (416 vs 640 px) and a CPU instead of a GPU. Our precision is close to theirs (0.786 vs 0.871) but our recall is lower (0.530 vs 0.654). Feng et al. [2] reach 0.840 with a larger model and about 37 times more images, which shows that more data helps. In all three studies recall is lower than precision.

7. User Interface Development and Deployment

The interface is a Streamlit web app (app.py). The model runs on the computer that starts the app, and users open it in a web browser, so it works on any device without installing anything.

Table 8. Interface features.

Component	Function
Image tab	Upload a photo; shows the original and the detected image side by side
Camera tab	Takes a photo with the camera of the device, including mobile phones
Video tab	Runs detection on an uploaded video, frame by frame
Live webcam tab	Live detection from the laptop webcam with FPS
Warning banner	Red warning that names the missing PPE; green when everyone is safe
Sidebar	Model selection, confidence slider, class filter

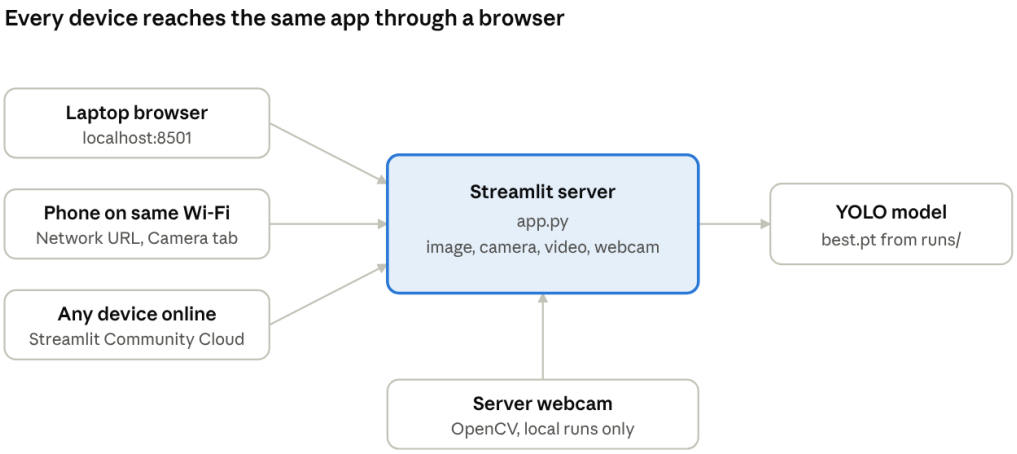


Figure 5. Ways to open the app on different devices.

Deployment. On a laptop the app starts with *python -m streamlit run app.py* (Windows, macOS or Linux). With *--server.address 0.0.0.0*, phones and tablets on the same Wi-Fi can open it using the network address it prints. For online access, the GitHub repository can be deployed on Streamlit Community Cloud using requirements.txt and packages.txt. The live webcam tab works only when the app runs on the laptop;

online, the Camera tab uses the visitor's own camera.

Code quality. Each step is a separate script (data_cleaning.py, train.py, evaluate.py, app.py) with comments and command-line options, and the dependencies are listed in requirements.txt.

8. Conclusion and Future Work

We built a working real-time PPE detection system: the dataset was cleaned and checked, a YOLOv8n model was trained (0.600 mAP@0.5 at about 25 FPS on a CPU), compared with state-of-the-art models, and deployed as a web app that works on laptops and phones. Future work: train longer on a GPU, try YOLO11n and YOLOv8s, add photos of normal caps to reduce cap/hardhat confusion, and track workers across video frames.

References

[1] M. Ciftci, M. U. Turkdamar and C. Ozturk, "Leveraging YOLO Models for Safety Equipment Detection on Construction Sites," *Journal of Computing Theories and Applications*, vol. 1, no. 4, pp. 492-506, 2024, doi:10.62411/jcta.10453.

[2] R. Feng, Y. Miao and J. Zheng, "A YOLO-based intelligent detection algorithm for risk assessment of construction sites," *Journal of Intelligent Construction*, vol. 2, no. 4, 9180037, 2024, doi:10.26599/JIC.2024.9180037.

[3] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu and J. Chen, "DETRs Beat YOLOs on Real-time Object Detection," in *Proc. IEEE/CVF CVPR*, 2024, pp. 16965-16974, arXiv:2304.08069.

[4] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han and G. Ding, "YOLOv10: Real-Time End-to-End Object Detection," in *NeurIPS*, 2024, arXiv:2405.14458.

[5] C.-Y. Wang, I-H. Yeh and H.-Y. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," arXiv:2402.13616, 2024.

[6] R. Khanam and M. Hussain, "YOLOv11: An Overview of the Key Architectural Enhancements," arXiv:2410.17725, 2024.

[7] Ultralytics, "YOLOv5, YOLOv8 and YOLO11 documentation," docs.ultralytics.com/models.

[8] Roboflow, "Construction Site Safety" dataset, Kaggle: snehilsanyal/construction-site-safety-image-dataset-roboflow.

[9] Streamlit, open-source Python framework for data applications, streamlit.io.